Meowtion

On-device behaviour recognition for early feline health insight

Jerome Graves and Rose Delcour-Min

Revision 3.0 • June 2026

Abstract. Cats are experts at hiding illness and pain. By the time something looks obviously wrong it is often already advanced, because the first warning is rarely a dramatic symptom; it's a shift in routine: eating a little less, drinking a little more, grooming and moving less than before. No owner can watch for that continuously, and memory makes a poor baseline. Meowtion turns the collar itself into that baseline. A lightweight, battery-powered collar classifies activities on the device in real time with motion and audio capture, using a confidence-gated cascade of two quantised neural networks. A motion model runs continuously, and only when movement alone is ambiguous does a short audio model step in to confirm. Raw audio never leaves the collar; only the result is relayed, over Bluetooth Low Energy through a home base station, to a live dashboard that surfaces the habit changes that matter for health. Critically, the activities are not fixed in firmware: owners define what to recognise, and a cloud trainer delivers new models back to the collar over the air. The outcome is a complete system, demonstrated end to end on real hardware, and a blueprint for private, on-device health monitoring that is a programmable platform, not a single-purpose gadget.

Contents

List of Figures

List of Tables

1 Introduction

1.1 Motivation

Cats are evolved to conceal illness. A cat that feels unwell will mask it for as long as it can, so by the time a problem is obvious to an owner it is often already advanced. The early signal is rarely a dramatic symptom; it is a quiet shift in routine. A cat that begins to drink noticeably more, eats less, stops grooming, or sleeps through activity it used to enjoy is communicating a change. The difficulty is that no person watches their cat continuously, and day-to-day memory is a poor baseline. The change is real but invisible.

Meowtion makes that change visible. It observes the behaviours that make up a cat's routine, continuously and unobtrusively, and surfaces the trend to the owner. The goal is explicitly *not* to diagnose; it is to notice a routine change early enough that the owner can act on it, rather than only seeing the crisis. The clinical literature on feline medicine is consistent on the point that appetite, thirst, grooming and activity are among the first things to shift when a cat becomes unwell, and that these shifts are easy to miss without a continuous, objective baseline.

1.2 Approach

The system is a wearable plus a small fixed gateway. A lightweight, battery-powered collar carries the sensors and the intelligence: it classifies behaviour on the device itself, in real time, and never streams raw sensor data off the cat. It reports only a compact result (for example, "eating, high confidence") over Bluetooth Low Energy (BLE) to a mains-powered base station in the home. The station bridges BLE to the internet over Wi-Fi and forwards results to a cloud backend, which drives a hosted dashboard where the owner sees their cat's activity and trends live, with the longer-run habit changes that matter for health flagged automatically.

Figure 1: A cat wearing the Meowtion collar.

1.3 Design principles

Four commitments shape the whole system and recur throughout this document.

Privacy by construction.
Classification happens on the collar. The microphone is used only to help recognise behaviour

on-device; raw audio is processed and discarded, never recorded or transmitted in normal operation. Each owner's data is scoped to their own account by the backend's security rules (section 13).

Trainable, not fixed-function.
The recognised behaviours are defined by the owner, not baked into firmware. Owners capture and label example clips, a cloud trainer produces tiny models, and those models are delivered back to the collar over the air. Meowtion is a platform for recognising any pet behaviour, not a three-trick gadget (section 8).

Intelligence at the edge.
Inference runs on a microcontroller on the cat, not in the cloud. This keeps the radio uplink tiny (a classification result, not a sensor stream), keeps average power low enough for a collar-sized battery, and removes the cloud from the sensing loop entirely (section 7).

Health-first analytics.
The dashboard is built around longitudinal habit tracking and baseline comparison, not just a live read-out, so that a sustained change in a key habit is surfaced as an insight rather than buried in raw events (section 10).

1.4 Contributions

Considered as an engineering artefact, Meowtion's notable elements are: a *confidence-gated, dual-model cascade* that runs an always-on motion classifier and invokes a short audio classifier only to resolve genuinely ambiguous behaviours (eating and drinking look alike to an accelerometer but not to a microphone), so audio is a deployed disambiguation signal rather than only a training aid; a *label-agnostic* pipeline in which the behaviour set is data-defined end to end, from the dashboard's action list through the cloud trainer to the index-based on-collar classifier; an *over-the-air model path* that delivers quantised models to a Bluetooth-only wearable through the station; and a privacy and multi-tenancy model in which battery devices hold only a scoped, revocable token and never the owner's credentials.

1.5 Scope and structure of this document

This document is the complete technical reference for Meowtion as built. It is organised so that a reader can either follow it top-to-bottom or jump to one tier. Section 3 gives the system-level picture, the component responsibilities, the two operating modes, and the end-to-end data flows, and is the right place to start. The detailed chapters then proceed bottom-up: the physical hardware (section 4); the firmware on each of the two microcontrollers (sections 5 and 6); the on-device AI (section 7); the training pipeline that produces and delivers models (section 8); the cloud backend (section 9); the dashboard (section 10); the communication protocols that tie the tiers together (section 12); and the security and privacy model (section 13). Section 15 records known limitations and planned work.

2 Requirements and design constraints

This section states the requirements the system was designed to meet and the constraints that shaped the design, so the chapters that follow can be read as the response to them. The final column of each table points to the section where the requirement is addressed.

2.1 Functional requirements

Table 1: Functional requirements.

ID	Requirement	Addressed in
FR-1	Recognise a cat's everyday behaviours (eating, drinking, activity, rest, grooming) from a collar-worn sensor.	section 7
FR-2	Perform classification on the wearable itself, in real time, without streaming raw sensor data off the cat.	section 7
FR-3	Let the owner define the behaviour set; do not hard-code it in firmware.	section 8
FR-4	Provide a closed training loop (capture, label, train, deploy) driven entirely from the dashboard, with no local toolchain.	section 8
FR-5	Deliver trained models to the collar over the air, with no firmware reflash.	sections 6 and 12
FR-6	Present the owner a live view and a longitudinal history with health-oriented analytics.	section 10
FR-7	Support multiple owners, each seeing only their own cats and devices.	section 9
FR-8	Offer a public, read-only way to explore the product without an account.	section 10

2.2 Non-functional requirements

2.3 Constraints

Radio.
The collar is Bluetooth Low Energy only; BLE is low-power but short-range and is not an internet transport, so a mains-powered station is required as the gateway (section 3).

Compute.
On-device inference runs on a Cortex-M4F with a few hundred kilobytes of RAM and no operating-system heap budget to spare, so models are int8 and arenas are statically sized (section 7).

Connectivity.
The station is provisioned with credentials but never logs in as a user; it authenticates to the cloud with a single revocable token (section 6).

Hosting.
The dashboard is hosted on a managed Python platform (Streamlit Community Cloud), which does not expose server-side cookies, shaping how the session is read (section 10).

These requirements and constraints are referenced again in section 14, which reports the extent to which each is currently met.

3 System architecture and data flow

This section is the map for the rest of the document. It introduces the four tiers and what each is responsible for, the two operating modes the system runs in, and then walks the four end-to-end data flows that together cover everything the system does: live monitoring, training capture, device onboarding, and model delivery.

Table 2: Non-functional requirements and quality attributes.

ID	Requirement	Addressed in
NFR-1	**Power.** The collar must run for a useful time on a collar-sized cell, which precludes Wi-Fi on the wearable and bounds the duty cycle of the audio path.	sections 4 and 7
NFR-2	**Footprint.** Inference must fit the microcontroller's RAM alongside the BLE stack.	section 7
NFR-3	**Privacy.** Raw audio must never leave the collar in normal operation; data must be scoped per owner.	section 13
NFR-4	**Credential safety.** Battery devices must not hold the owner's password; device authorisation must be revocable.	section 13
NFR-5	**Reproducibility.** A third party must be able to build and run the system from the repository without contacting the authors.	section 9
NFR-6	**Openness.** Hardware designs and code are released under open licences.	section 4

3.1 Topology

Meowtion is a four-tier system, fig. 2: the **collar** on the cat, the **station** in the home, the **cloud** backend, and the owner's **dashboard**. The defining constraint is that the collar is BLE-only and battery-powered: it has no route to the internet of its own. The mains-powered station exists to bridge that gap, giving the collar a cheap, always-on Bluetooth peer and a Wi-Fi path to the cloud. Everything above the collar is conventional web infrastructure; everything at the collar is shaped by power and radio budget.

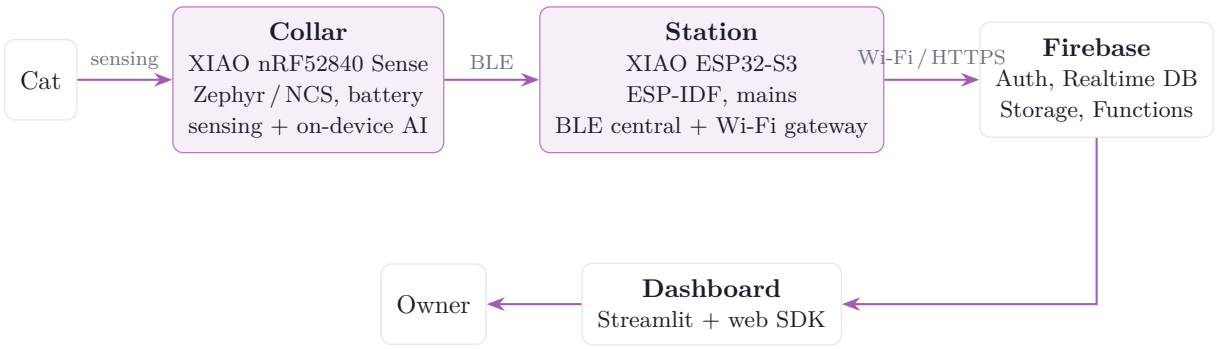


Figure 2: End-to-end topology. The collar is BLE-only, so the always-on station is its gateway to the cloud. The dashboard is a separate client of the same backend.

3.2 Component responsibilities

3.3 Operating modes

A station runs in one of two modes at a time, selected per station from the dashboard. The mode governs what crosses the BLE link and what the station does with it.

Production mode

The everyday monitoring mode. The collar classifies on-device and advertises only the result; the station reads that result and relays it to the cloud. No audio or raw motion ever leaves the collar. This is the privacy-preserving default.

Table 3: What each tier is responsible for.

Tier	Responsibility
Collar	Sense (IMU + microphone), classify behaviour on-device, advertise the result as BLE telemetry, and, when a station subscribes, stream paired audio + IMU clips for training and accept models over the air. Holds no credentials.
Station	Scan for and relay registered collars to the cloud; capture and upload training clips; fetch trained models from Storage and push them to the collar over BLE. Authenticates to the cloud with a per-device token, not a login.
Cloud	Firebase: authenticate owners, store the live data and history (Realtime Database), store training clips and trained models (Storage), and run server-side logic (Cloud Functions for training and authenticated clip upload). Enforces per-owner isolation.
Dashboard	The owner-facing web app: a live current-state view and longitudinal health analytics, plus a static front door for sign-in and device registration and a dev console for data capture and training.

Training mode
The data-collection mode. While a registered collar is in range, the station subscribes to the collar's audio service; the collar streams paired audio + IMU clips, which the station slices, uploads, and the owner labels and trims in the dev console. This is how a new behaviour is taught. It is used deliberately and temporarily, not continuously.

3.4 End-to-end data flows

The system's behaviour is fully described by four flows. Each is given below as the concrete sequence of steps across the tiers; the wire formats referenced are specified in section 12 and the security rules in section 13.

Flow 1 — Live monitoring (production). The steady state of a deployed system.

1. The collar samples its IMU continuously and runs the on-device cascade (section 7); when the motion result is ambiguous it briefly samples the microphone and runs the audio model to confirm.
2. The collar encodes the outcome (class index, confidence, step count, battery) into an 8-byte BLE telemetry advertisement (section 12). Its BLE address identifies which collar it is, so no identifier is carried in the payload.
3. The station, scanning continuously, hears the advertisement from a *registered* collar, timestamps it, maps the class index to the owner's label set, and writes the current state plus an event record to the Realtime Database over authenticated HTTPS.
4. The dashboard, reading the same database, refreshes the live card within seconds and folds the new event into the history and health analytics (section 10).

Flow 2 — Training capture. How labelled data is collected to teach a behaviour.

1. In the dev console the owner puts a station into training mode and selects the action being recorded (the "live label").
2. The station connects to the in-range collar and subscribes to its audio characteristic. The collar streams continuously in 100 ms frames, each frame a header followed by an audio payload and a time-aligned IMU payload (section 12).

Figure 3: Flow 1, live monitoring. The collar advertises an 8-byte telemetry packet; the station maps the class index to a label and writes the live state and events to the database, which the dashboard reads.

3. The station concatenates frames, slices them into fixed-length clips, and POSTs each clip's bytes to the `upload_clip` Cloud Function with its device token. The function verifies the token's owner and writes the clip to that owner's Storage area; the station records the clip's metadata (label, paths, IMU presence) in the database.
4. The owner reviews clips in the dev console: trims the waveform, confirms or changes the label, and discards bad takes. The labelled clips are the training set for Flow 4.

Flow 3 — Device onboarding. How a station and collar are registered to an owner, performed once per device.

1. The owner signs in on the static front door and connects the station over USB using the browser's Web Serial API. The page mints a random per-device *token*, records it under the owner's account (`deviceTokens/<token>/owner`) and as a station device entry, and hands the station its Wi-Fi credentials, the owner id, and the token over the serial link. The station stores these in non-volatile storage; the token is its sole credential.
2. The station comes online, syncs time, and begins scanning. It publishes any unregistered collars it hears to its `seen` list.
3. The owner registers a heard collar with one tap in the dashboard. The collar then appears as a device on the account and is relayed by the station. Deleting the token revokes the device.

Flow 4 — Model training and over-the-air delivery. How labelled clips become a model running on the collar.

1. The owner triggers training from the dev console. The `train` Cloud Function reads the account's labelled clips, derives the class set from the labels present, trains an int8 IMU model and an int8 audio model, uploads them to Storage under `models/<uid>/`, and bumps the account's model version (section 8).
2. The station learns a new version is available (the version is mirrored into its own device config), downloads each model from Storage, and, over a brief dedicated BLE connection, streams it to the collar with a CRC-checked begin/data/end protocol (section 12).
3. The collar stages the model into its dedicated flash partition, verifies the CRC, loads it, and switches from advertising simulated state to advertising real classifier output. The whole path requires no firmware reflash.

3.5 Multi-tenant data model

Meowtion is multi-tenant. Each owner creates an account and registers their own station(s) and collar(s); all of an owner's data lives under `users/<uid>/` in the Realtime Database, and the database and storage rules scope every read and write to the owning account (sections 9 and 13). Isolation is enforced by the backend, not the client. Battery devices never hold a login: a station authenticates with its scoped, revocable token, and a collar holds nothing at all (the

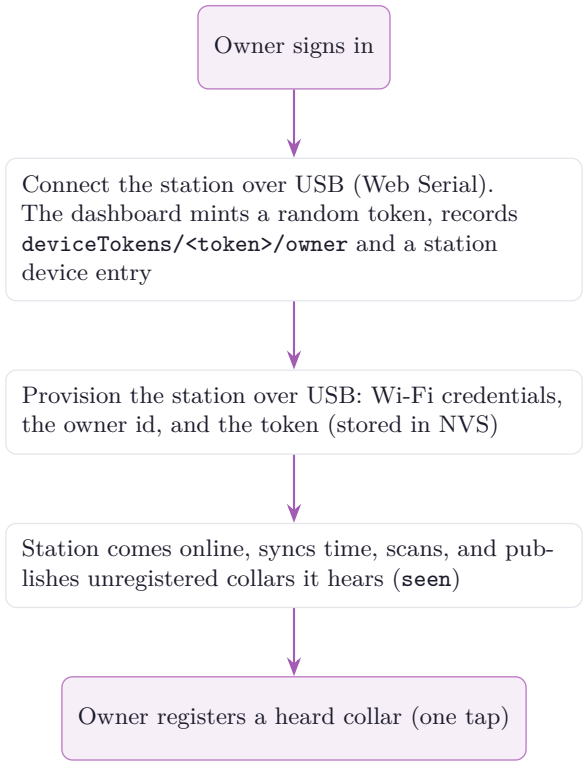


Figure 4: Flow 3, connecting devices. A station is paired over USB and provisioned with a scoped token; once online it surfaces nearby collars, which the owner registers with one tap.

station relays it). A single public, read-only demo account, designated in server-side config, lets anyone explore a populated dashboard without signing up.

3.6 Technology summary

Table 4: Technology at each tier.

Tier	Stack
Collar	Seeed XIAO nRF52840 Sense; Zephyr / nRF Connect SDK (v3.3.1); TensorFlow Lite Micro with CMSIS-NN kernels
Station	Seeed XIAO ESP32-S3; ESP-IDF; NimBLE (BLE central) + Wi-Fi station
Cloud	Firebase: Authentication, Realtime Database, Cloud Storage, Python Cloud Functions (2nd gen)
Dashboard	Streamlit (Python) + Firebase web SDK (JavaScript); hosted on Streamlit Community Cloud
Training	TensorFlow / Keras, executed server-side in a Cloud Function; int8 post-training quantisation to TensorFlow Lite

4 Hardware

The build is two devices: the collar worn by the cat and the station plugged in at home. There is no custom PCB. Each device is a Seeed Studio XIAO module in a 3D-printed shell, chosen for the small footprint, integrated radios, hand-solderable castellated pads, and, on

the collar, on-module sensors. The complete parts list, print files and assembly steps live in `hardware/README.md`; this section summarises the design and the reasoning behind it.

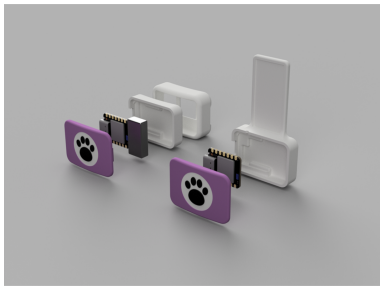
4.1 Collar

The collar is built around the Seeed XIAO nRF52840 Sense. The nRF52840 provides a Bluetooth Low Energy radio and enough flash and RAM to run quantised neural-network inference on its Cortex-M4F. Critically for Meowtion, the *Sense* variant carries both sensors the system needs on the module itself, a six-axis IMU (LSM6DS3TR-C) and a PDM MEMS microphone, so the collar needs no external sensor breakout and stays small and light.

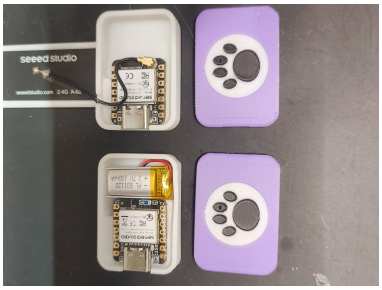
Table 5: Collar bill of materials (one per cat).

Part	Qty	Role
Seeed XIAO nRF52840 Sense	1	SoC (BLE), on-module IMU and PDM microphone; runs inference
1S LiPo cell, 3.7 V 100 mAh, with protection (BMS)	1	Powers the wearable; soldered to the BAT pads. A cell with a built-in protection circuit is required
20 mm hydrophobic PTFE filter membrane	1	Sits over the mic hole: passes sound, keeps water and dust out
10 mm quick-release safety collar	1	Standard strap; the silicone skin mounts to it
3D-printed shells + TPU skin	1	PETG front and back shells; a flexible TPU skin holds the sealed assembly on the strap

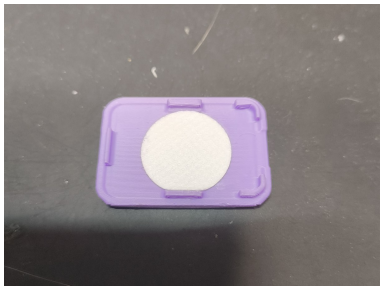
The IMU supplies the motion stream that is always classified; the microphone supplies the short audio window used to disambiguate behaviours that look alike by motion but differ by sound (section 7). The microphone port sits behind the PTFE membrane in the front shell; correct membrane placement matters for audio quality and water resistance and is documented in the hardware guide. The collar runs on its own 1S cell, charged through the XIAO's onboard divider.



(a) Exploded view.



(b) Assembled internals.



(c) PTFE membrane over the mic hole.

Figure 5: Collar and station hardware. Full parts, print files, and assembly steps are in `hardware/README.md`.

4.2 Station

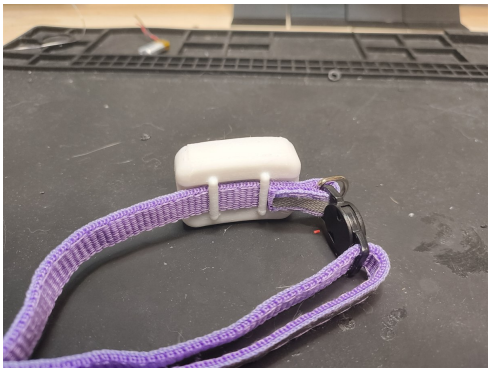
The station is a Seeed XIAO ESP32-S3. The ESP32-S3 has both Wi-Fi and BLE, so a single module acts as the BLE central that listens to the collar and the Wi-Fi client that reaches the

cloud. It is mains-powered over USB-C, so it can scan and stay online continuously without the power constraints of the wearable. The station carries no sensors of its own; its job is to hear the collar, capture training clips when asked, relay results to the cloud, and push models to the collar.

4.3 Enclosure and assembly

The front shell is shared between the collar and the station; the back shells and the collar's TPU skin are per-device. The collar shells are PETG for rigidity and the skin is flexible TPU; the station back shell is any rigid filament. The collar is assembled by seating the board and cell in the back shell, fitting the PTFE membrane over the mic hole, bonding the shell halves with water-resistant glue, and slipping the sealed unit into the TPU skin on a quick-release strap. The station is simply seated, glued closed, and plugged in by the bowl. The hardware designs are released under CERN-OHL-S v2 (strongly reciprocal open hardware).

(a) Front, on the strap.



(b) Back, on the strap.

Figure 6: The finished collar on a quick-release strap.

4.4 Why split the system this way

The collar must be small, light, and run for a useful time on a small cell, which rules out Wi-Fi on the wearable: its power draw is incompatible with a collar-sized battery. BLE is low-power but short-range and is not an internet transport. The fixed, mains-powered station resolves both problems: it gives the collar a cheap always-on BLE peer and a Wi-Fi path to the cloud. The split also keeps the privacy boundary clean, as the heavy and sensitive processing (including all audio) stays on the collar and only a classification result crosses to the station.

5 Collar firmware

The collar runs on Zephyr [1] via the nRF Connect SDK (NCS v3.3.1), targeting the board `xiao_ble/nrf52840/sense`. It is a connectable BLE peripheral that senses on the cat, advertises a compact telemetry packet, and, when a station connects and subscribes, streams paired audio + IMU clips for training. It never touches Wi-Fi or the internet, which keeps it small and low-power and lets it always run on its own battery. The build is driven by `build.ps1`, which locates the local NCS toolchain, runs `west build -b xiao_ble`, and copies out the UF2.

5.1 Responsibilities

The firmware has four jobs, driven from the main loop and BLE callbacks:

1. Sample the IMU continuously (and the microphone when needed) and run the classifier cascade.
2. Advertise the result as an 8-byte BLE telemetry packet for the station to read (section 12).
3. In training mode, stream paired audio + IMU frames to a subscribed station for capture.
4. Receive new models over the air and swap them in safely.

5.2 Source layout

Table 6: Collar firmware modules (`firmware/collar/src/`).

File	Responsibility
`main.c`	Boot and the telemetry loop; orchestration only
`ble.c`	Advertising, identity, the audio GATT service, notifications
`audio.c`	PDM microphone capture, decimated to 8 kHz μ-law
`audio_codec.h`	μ-law codec and the model-input audio representation
`streaming.c`	Decoupled reader/sender threads that assemble and send clip frames
`imu.c`	LSM6DS3TR-C continuous sampler
`battery.c`	1S LiPo charge state via the onboard divider
`classifier.c`	The confidence-gated cascade policy (IMU first, audio confirms)
`tflm_classifier.cpp`	TensorFlow Lite Micro backend for the cascade
`model_loader.h`	Runtime model-install API (`clf_set_imu_model` / `clf_set_audio_model`)
`production.c`	Rolling IMU window; runs inference and writes real telemetry
`ota.c`	OTA receive state machine; stages models to flash and defers the swap

The classifier is split deliberately. `classifier.c` defines the cascade policy in plain C with *weak* hooks for model-presence queries and inference calls; `tflm_classifier.cpp` provides the *strong* `extern "C"` definitions that override them when the TensorFlow Lite Micro backend is compiled in. This keeps the policy readable and separable from the heavy C++ inference, and it means the firmware links and runs even with the backend or any model absent.

5.3 Streaming and threading

Capturing training data is the collar's most timing-sensitive path: PDM audio arrives in a steady stream and must be paired with IMU samples and pushed over BLE without dropping frames. The reader and sender are therefore decoupled (`streaming.c`): one side assembles 100 ms frames from the audio and IMU sources, the other side sends them over the audio characteristic, so a momentarily slow radio cannot stall capture. The collar negotiates a large ATT MTU (247 bytes) and uses the 2 Mbit PHY to keep throughput up.

5.4 Model storage and flash layout

Models are not linked into the firmware image. They live in a dedicated flash partition so a new model can be written over the air without reflashing. The partition map is fixed in `pm_static.yml` to stop the partition manager from silently regrowing the application region over the model storage:

Table 7: Collar flash partitions (`pm_static.yml`).

Partition	Address / size	Contents
`app`	`0x27000` / `0x85000`	Application code (fixed size)
`models_storage`	`0xAC000` / `0x40000` (256 KiB)	Header plus the IMU (slot 0) and audio (slot 1) model blobs
`settings_storage`	`0xEC000` / `0x08000`	NVS settings
`uf2_partition`	`0xF4000` / `0x0C000`	UF2 bootloader (unchanged)

5.5 Build configuration

Enabling on-device inference meant pulling TensorFlow Lite Micro into the NCS workspace (an optional, normally-excluded west module) and turning on C++17 and the TFLM and CMSIS-NN options. Two stability fixes are baked into the configuration: TFLM's `AllocateTensors()` and `Invoke()` overflow Zephyr's small default stack, so the main stack is raised; and the model storage uses `stream_flash` to lazily erase pages during an OTA write.

```
CONFIG_CPP=y
CONFIG_STD_CPP17=y
CONFIG_TENSORFLOW_LITE_MICRO=y
CONFIG_TENSORFLOW_LITE_MICRO_CMSIS_NN_KERNELS=y
CONFIG_MAIN_STACK_SIZE=16384      /* AllocateTensors()/Invoke() overflow the ~2 KiB
     default */
CONFIG_STREAM_FLASH_ERASE=y       /* lazily erase model pages during OTA */
CONFIG_BT_L2CAP_TX_MTU=247        /* large MTU for audio streaming + OTA chunks */
CONFIG_BT_CTLR_PHY_2M=y           /* 2 Mbit PHY for throughput */
```

Listing 1: Inference-related `prj.conf` settings (collar).

Because committing a model touches a lot of stack, the OTA receive path does not load the model from inside the BLE-RX callback. It stages the bytes to flash, then defers the verify-and-swap onto the main thread (section 12), so a model can be installed without starving the BLE stack.

5.6 Development console and flashing

The XIAO nRF52840 has no debug COM port, so flashing is always a UF2 drive-copy: double-tap RESET to mount the bootloader drive, then re-run the build script to copy the UF2 across. For development the firmware brings up a USB CDC ACM console on the *legacy* USB device stack (the newer NEXT stack's CDC console silently drops output on this board). On that console the collar prints a `MEOW> collar id=...` banner and a periodic `state= steps= batt=` line; the `id=` banner is what the dashboard reads over Web Serial to register the collar.

5.7 Current state

Telemetry state, activity and step count are currently produced by a dwell-based behaviour state machine (real battery is wired) and switch to live classifier output once a model is loaded; until then the cascade reports `UNKNOWN` and the firmware runs model-free and safe (section 7). The cascade itself already runs at capture time, on the IMU window paired with each clip, so the inference path is exercised before any model ships.

6 Station firmware

The station runs on ESP-IDF [2] on the XIAO ESP32-S3. It is a BLE central that hears registered collars and a Wi-Fi gateway that relays them to the owner's Firebase account. It also captures short audio + IMU clips for training and uploads them, and it delivers trained models to the collar over BLE. It has no Firebase login: Wi-Fi credentials, the owner id, and a per-device token are provisioned over USB serial and stored in non-volatile storage (NVS), and the database authorises the station because that token is registered to the owner.

6.1 Source layout

Table 8: Station firmware modules (`firmware/station/main/`).

File	Responsibility
`main.c`	Boot and the roughly one-second orchestration loop
`config.c`	Provisioned credentials, NVS storage, USB-serial provisioning
`board.c`	Power-source detection (USB vs battery)
`wifi.c`	Wi-Fi station bring-up and time synchronisation
`cloud.c`	The Firebase Realtime Database / HTTP layer (the only HTTP in the firmware)
`weather.c`	Local weather from the Open-Meteo feed
`ble.c`	The whole NimBLE collar subsystem (scan, relay, audio capture) behind `ble_service()`
`ota.c`	OTA model delivery: pushes trained `.tflite` models to the collar over BLE

6.2 Provisioning

Provisioning is a single JSON line sent over USB serial; the dashboard's "Connect a device" form does this automatically over Web Serial, but it is plain text and can be sent by hand for debugging:

```
{"ssid":"..","pass":"..","owner":"<uid>","token":"<random>","name":"..","lat
    ":50.8,"lon":-0.1}
```

Listing 2: Provisioning line (USB serial).

The station stores these in NVS and can be re-provisioned at any time from the dashboard, or wiped with `idf.py erase-flash`. The token is the station's only credential; deleting it from the owner's account revokes the device immediately. Repository secrets such as `secrets.h` are excluded by `.gitignore`, so no live credential is tracked.

6.3 Relay and clip upload

In production mode the station scans continuously, hears the telemetry advertisements of registered collars, maps the compact result (mode, class index, confidence) onto the cat's record, and writes both a live `current` state and discrete activity events as episodes begin and end (`cloud.c`). Events carry the model version and class index rather than a resolved name, so the dashboard can map them to labels even as the label set changes (section 9). In training mode it connects to an in-range collar, captures clips, and POSTs each clip's bytes to the `upload_clip` Cloud Function with its device token; the function verifies the token's owner and writes the file to that owner's Storage area, since clients cannot write Storage directly. A manual capture path ignores the signal-strength gate, used to collect behaviours such as purring that happen while the cat rests rather than at the bowl.

6.4 Over-the-air model delivery

The station is the delivery vehicle for trained models, since the collar is BLE-only and never touches Storage. The wire protocol is fixed in the shared header `firmware/common/meow_ota.h`: the collar is the GATT server that stages a `.tflite` into flash, the station is the client that pushes it (section 12).

Version gate.
The `train` function bumps `users/<uid>/models/version` and mirrors it into this station's own config as `modelVer` (the station may read only its own `users/<uid>/devices/<token>/` subtree, not `users/<uid>/models`). The station keeps its last-pushed version in NVS; a push is due when the config version is higher.

Download.
For each slot it fetches the model from Storage at the public-read path `models/<uid>/imu_model.tflite` (IMU slot) or `models/<uid>/audio_model.tflite` (audio slot). A 404 means that slot has no model yet and is skipped.

Push.
It computes a CRC-32/IEEE over the bytes, then over a brief dedicated BLE connection writes `BEGIN`, streams the model to the data characteristic in write-with-response chunks (the chunk write-response is the flash-write flow control), and writes `END`. On the collar's `OK` it records the new version in NVS; on any error it leaves NVS unchanged and retries later.

A push runs only when a registered collar is in range and audio capture is idle, so model delivery never contends with the capture or relay path for the radio.

6.5 Robustness: scanning after an update

The station drives one BLE radio for several jobs, and the OTA path exposed a subtle defect. To push a model the station must cancel scanning and open a connection; the OTA path used its own GAP callback and originally never restarted scanning when it finished, so after an update the station went deaf and the dashboard reported the collar offline. The fix is an idempotent `ble_resume_scan()`, guarded on the discovery-active flag, that the OTA path calls on both completion and bail-out. It was verified on hardware.

6.6 Build and flash

The station builds with `idf.py build` and flashes over its USB serial port with `idf.py -p <COM> flash monitor` (the repository's VS Code task **Build+Flash Station** wraps this). It is verified building under ESP-IDF.

7 On-device AI

All classification happens on the collar, in real time, using TensorFlow Lite Micro [3] with the CMSIS-NN kernels [4] for the nRF52840's Cortex-M4F. The models are tiny int8 networks in the spirit of the TinyML approach [5]: small enough to fit a microcontroller's RAM and run within the collar's power budget, yet accurate enough to separate everyday behaviours.

7.1 Confidence-gated cascade

Two behaviours can look almost identical to a motion sensor while sounding completely different: eating and drinking both put the head down at a bowl with similar gross motion. Rather than

force a single model to separate them from accelerometry alone, Meowtion runs a confidence-gated cascade of two models:

1. The **IMU model** runs continuously on the motion stream. It is cheap and always on, and it carries most of the classification load.
2. When the IMU model is *unsure* (its top class falls below a confidence threshold), the short **audio model** is invoked to confirm. Audio is therefore a deployed disambiguation signal, not merely a training aid.

This keeps average power low, since the audio path runs only when it is needed to break a tie, while recovering accuracy that motion alone cannot provide. The cascade policy lives in plain C (`classifier.c`) and is independent of the inference backend, so the gating logic can be read and tuned without touching the TensorFlow Lite Micro code. Figure 7 shows the decision flow.

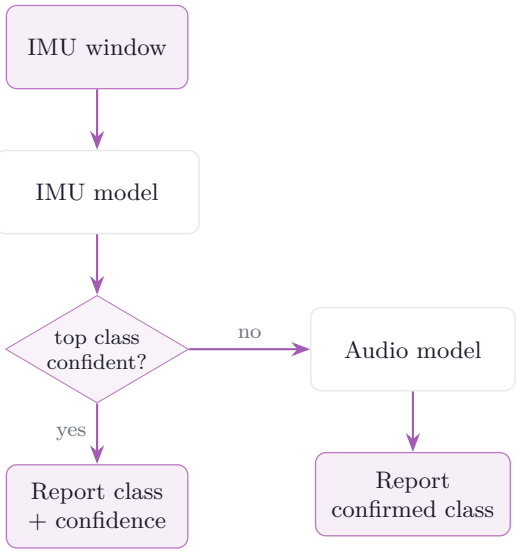


Figure 7: The confidence-gated cascade. The IMU model runs on every window; the audio model is invoked only when the IMU model's top class is below the confidence threshold. With no model loaded the cascade returns `UNKNOWN`.

7.2 Matching training and inference

The single most important correctness property is that the on-device input transform must mirror the transform used during training, exactly. A model trained on one representation and fed another at inference produces confident nonsense. Meowtion enforces the match on both paths:

- **Motion.** The rolling window is normalised per channel (subtract each channel's mean over the window, divide by the global maximum absolute value with a small epsilon) and quantised to int8 using the input tensor's own scale and zero-point.
- **Audio.** Inference audio is decimated and encoded to match the training representation: 8 kHz μ-law, produced by the same conversion used to prepare the training clips (section 12). A mismatch here was an early source of confident misclassification.

To avoid a large floating-point scratch buffer on a RAM-constrained part, the IMU path quantises directly from the int16 source in a few passes (per-channel mean, then global max-abs, then write int8) rather than materialising a float window, listing 3.

```
// pass 1: per-channel mean over the window
// pass 2: global max-abs of (sample - channel_mean)
// pass 3: int8 = round( x_norm / scale ) + zero_point,
//         where x_norm = (sample - channel_mean) / (maxabs + 1e-6)
// scale and zero_point are read from the input tensor itself.
```

Listing 3: Direct int16-to-int8 quantisation, mirroring training (sketch).

7.3 Runtime models and model-free safety

No model is compiled into the firmware. Each stage loads its `.tflite` at runtime (`clf_set_imu_model` / `clf_set_audio_model`), from the dedicated flash partition written over the air (sections 5 and 12). Until a model lands the engine runs model-free: the presence queries return false and the cascade returns `UNKNOWN`, so the firmware is safe to run with no model at all and simply advertises a simulated state until a real model is installed.

7.4 Memory budget

TensorFlow Lite Micro uses a fixed, statically allocated tensor arena per model (this firmware uses no heap). The arenas dominate the RAM cost of inference and are sized to the two networks. The values in table 9 are the starting budgets; each must be tuned to the real exported model, since an undersized arena makes `AllocateTensors()` fail and the stage report itself absent.

Table 9: Tensor arenas (initial budgets).

Model	Arena	Notes
IMU (slot 0)	16 KiB	Always-on motion classifier, a small temporal conv net
Audio (slot 1)	48 KiB	Larger 1-D conv over 8000 points; invoked only to confirm

Enabling inference initially pushed RAM use to roughly 94 %; eliminating the float-window scratch buffer (quantising directly from int16) brought it back into the low 80s, leaving headroom for the BLE stack and the raised main stack (section 5).

7.5 Operator set

The op resolver is a fixed `MicroMutableOpResolver`, so only the operators the models actually use are linked into the image. The Keras `Conv1D` layers export as `CONV_2D` wrapped in `EXPAND_DIMS` and `SQUEEZE`; the missing `EXPAND_DIMS` operator was the cause of an early "didn't find op" load failure, and the resolver now includes it alongside the pooling, fully-connected, reshape, softmax, and quantise/dequantise operators the networks need. Pinning the operator set this way keeps the binary small and makes a load failure a clear, diagnosable event rather than a silent fallback.

8 Training pipeline

Meowtion learns new behaviours through a closed loop that an owner drives entirely from the dashboard, with no local toolchain: capture, label, train, deliver, run. The loop is what turns the collar from a fixed-function sensor into a platform for recognising any behaviour the owner can demonstrate.

8.1 The loop

1. **Define behaviours.** The owner lists the actions to recognise (eat, drink, purr, scratch, litter-tray, and so on) in the dev console. This set drives both the clip-label dropdown and the trained model's outputs.
2. **Capture.** With a station in training mode, paired audio and time-aligned motion are recorded while a registered collar is in range and uploaded to Cloud Storage (Flow 2, section 3).
3. **Label.** The owner reviews clips in the dev console, sets each clip's action, trims dead space from the waveform, and discards poor examples.
4. **Train.** The `train` Cloud Function trains the models server-side from the labelled clips and writes the results back: the quantised `.tflite` blobs to Storage, and the new version and label list to the database.
5. **Deliver.** Each station watches a model-version key; when it changes the station downloads the new `.tflite` files and pushes them to the collar over the air (section 12).
6. **Run.** In production mode the collar classifies with the new model and the dashboard shows live, named behaviours.

Figure 8 shows how the training *data* moves through the system during the capture and label steps; fig. 9 (below) shows how the trained *model* moves back out.

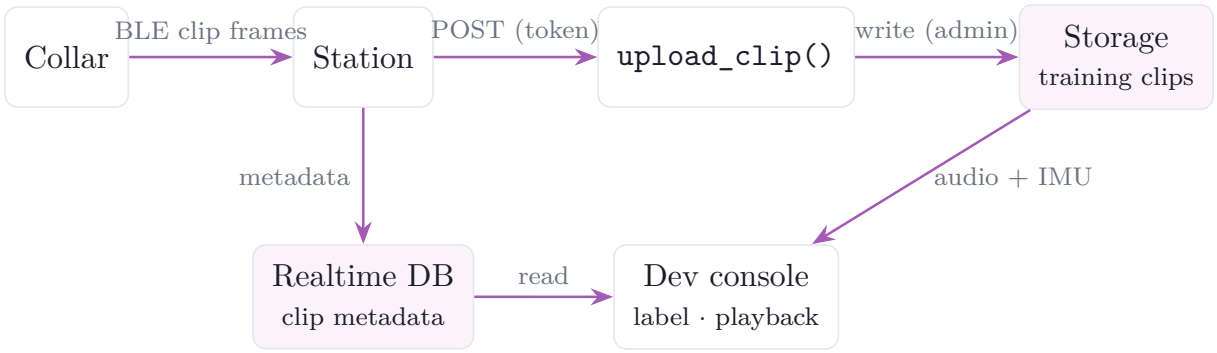


Figure 8: Training-data transport. The collar streams paired audio + IMU clip frames to the station, which slices them into clips and uploads the bytes through `upload_clip` to Storage and the clip metadata to the database; the dev console reads both to play back, label, and train on them.

8.2 The trainer

Training runs in the `train` Python Cloud Function, gated by a developer account (section 9). It reads the account's labelled clips (metadata from the database, audio and motion bytes from Storage), and derives the class set directly from the labels present, so the network's outputs follow whatever behaviours the owner defined. It then trains the two cascade stages independently:

- a small 1-D convolutional network over the motion windows (six IMU channels), and
- a larger 1-D convolutional network over the 8 kHz audio windows.

Each input is windowed with overlap and normalised exactly as the collar normalises it at inference, so the on-device transform and the training transform match (section 7). The trained Keras models are converted to TensorFlow Lite with full int8 post-training quantisation using a representative dataset, so the exported `.tflite` weights and activations are int8 and the input and output tensors carry the scale and zero-point the collar uses to quantise its own inputs. Figure 9 shows the whole path, from the training trigger to the model running on the collar.

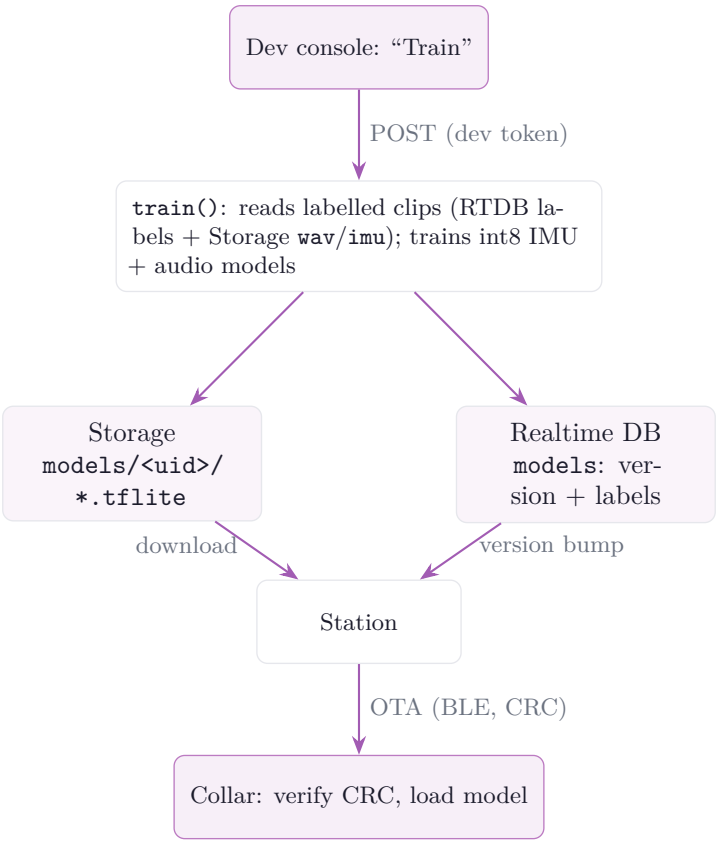


Figure 9: Training and over-the-air model delivery. `train()` reads the labelled clips, writes the quantised models to Storage and the new version and labels to the database, and the station detects the version bump, downloads the models, and pushes them to the collar.

8.3 Label-agnostic by design

The trainer is not hard-coded to a fixed set of actions; it learns whatever label set the owner defines. This is what makes Meowtion a platform rather than a fixed-function gadget: the same pipeline that recognises eating and drinking can be pointed at any distinguishable behaviour the owner can capture and label. The class set flows end to end as data, from the dashboard’s action list, through the trainer, to the index-based on-collar classifier, with names attached only at the edges.

8.4 Delivery contract

Training outputs are written to predictable locations so the station finds them without coordination: the model blobs land in Storage as `models/<uid>/imu_model.tflite` and `models/<uid>/audio_model.tflite`, the new version is mirrored into each of the owner’s stations’ configuration so their version gate fires, and the label list is stored in `users/<uid>/models` so the dashboard can map a class index back to a name across label-set changes (section 9).

8.5 Known limitation

A model trained on only two classes (for example, eat versus drink) has no way to represent “neither” and will confidently assign one of its two labels to an idle cat. The fix is to retrain with an explicit idle/other class; this is a data and modelling task, not a firmware change, and is tracked in section 15.

9 Cloud backend

The backend is Firebase [6]. It provides owner authentication, a Realtime Database for live state and history, Cloud Storage for training clips and model blobs, and Python Cloud Functions (2nd generation) for server-side logic. There is no custom server to operate, and the whole backend is self-contained in `app/firebase/`, deployed with `firebase deploy` from that folder.

9.1 Authentication

Owners sign in with Firebase Authentication (email/password or Google). The collar and station do *not* authenticate as users: a station carries a scoped per-device token, and a collar carries nothing (the station relays it). This keeps the credential surface on battery devices to a single revocable token (section 13).

9.2 Backend data flow

Figure 10 shows how the tiers exchange data in normal operation. The dashboard reads the database, both a small per-cat live read and a cached history read (section 10); the station writes telemetry and events to the database and uploads training clips to Storage through the `upload_clip` function; and the station relays the collar and delivers models to it over BLE. Clients never write Storage directly (section 13). The training path that produces those models is shown separately in fig. 9.

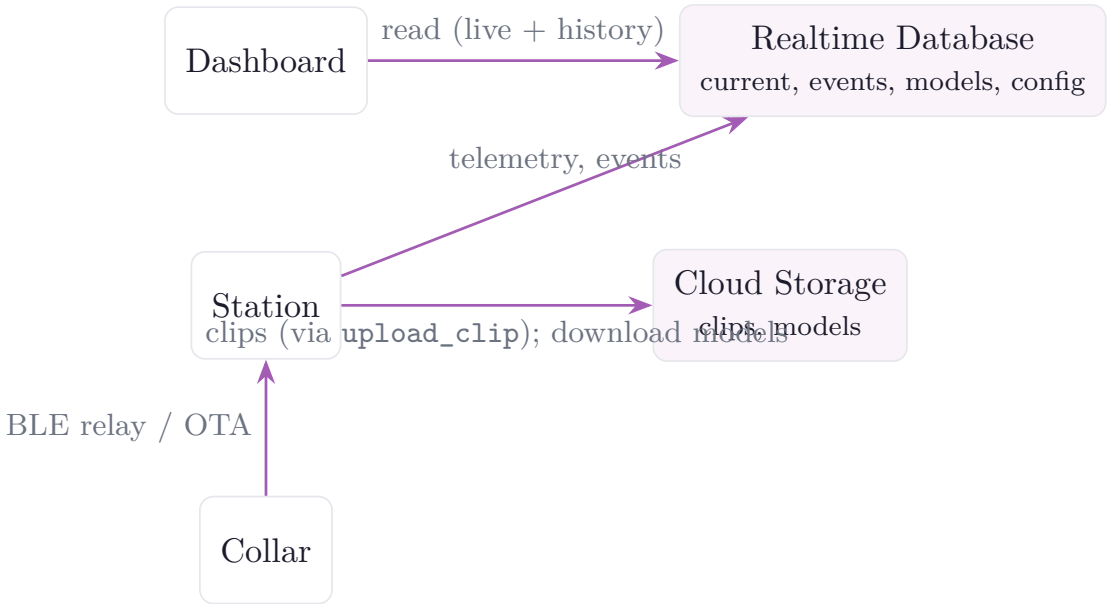


Figure 10: Backend data flow in normal operation. The dashboard reads the database; the station writes live state and events and uploads clips; the collar is relayed and updated through the station.

9.3 Realtime Database schema

State and history live in the Realtime Database, keyed by owner so the security rules can scope every read and write to the owning account. Table 10 lists the main paths.

Events store the model version and class index rather than a pre-resolved name. The dashboard maps the index to a label using that version's label list, so historical events stay correct even as the owner changes their behaviour set.

Table 10: Realtime Database paths.

Path	Contents
`users/<uid>/devices/<token>`	A registered station (`type:station`) with its relayed `cats/<catId>` subtree, or a registered collar entry
`.../cats/<catId>/current`	The cat's live state (version, class, confidence, steps, battery)
`.../cats/<catId>/events`	Discrete activity events (carry model version + class index)
`.../devices/<station>/weather`	Current and recent local weather for the station's location
`users/<uid>/models`	Current model version, status, and the label list
`users/<uid>/actions`	The behaviours to recognise (managed from the dev console)
`deviceTokens/<token>/owner`	Maps a device token to its owning account
`config/demoOwner`	UID whose data backs the public read-only demo
`config/devAccounts/<uid>`	Accounts allowed to use the dev/training tools

9.4 Cloud Storage

Storage holds the training clips (audio plus aligned motion) and the trained model blobs, in an owner-scoped layout: clips at `training/<owner>/<collar>/<ts>.{wav,imu}` and models at `models/<uid>/{imu,audio}_model.tflite`. Training clips are private to the owner and are deliberately *not* world-readable, even in the demo, because home audio must never be public. Model blobs are public-read so the token-only station can fetch one to push to the collar, and are written only by the trainer.

9.5 Cloud Functions

Two functions provide the only server-side logic. Both run on the function's ambient service account, so no service-account key is checked into the repository.

`train`
: Server-side model training, `POST` with a developer account's Firebase ID token and gated additionally by `config/devAccounts/<uid>`. It reads the account's labelled clips, trains the int8 IMU and audio models (section 8), uploads them to `models/<uid>/`, updates `users/<uid>/models`, and mirrors the new version into each of the owner's station configs to trigger delivery.

`upload_clip`
: Authenticated clip upload for the token-only station. The station has no login, so it cannot satisfy the Storage rules directly; it POSTs the clip bytes here with its device token, the function verifies `deviceTokens/<token>/owner`, and writes the file as administrator into that owner's Storage area. This lets the Storage rules deny all direct client writes while still letting an authorised station upload.

9.6 Demo data simulator

So that the public demo always presents a populated, believable dashboard, a scheduled Cloud Function maintains a fully simulated companion collar ("Purrminator") on the demo account. It is a standalone device that needs no station and is always online. On first run it backfills roughly six months of realistic activity history, weighted by time of day so the pattern matches an average cat (crepuscular, busy around dawn and dusk, resting overnight), with matching daily weather from a free historical feed. Thereafter it wakes on a short schedule and commits each

activity as an event at the moment that action ends, so new events arrive at naturally varying intervals. The simulator only ever writes the demo account's own data and never touches a real collar.

9.7 Weather context

Local weather is incorporated so the dashboard can read a habit change against its environment, since a warm spell that increases drinking and suppresses appetite should not be mistaken for illness. The station polls a keyless forecast feed (Open-Meteo) for its provisioned location and writes current and recent weather under its device record; the demo simulator uses the same provider's historical archive to backfill weather alongside the simulated activity.

9.8 Configuration as a trust boundary

The `config` subtree is readable by clients but not writable by them; it is maintained from the Firebase console. This lets the system publish facts the clients need (which account backs the demo, which accounts are developers) without letting a client grant itself privileges.

10 Dashboard

The owner-facing application is a Streamlit [7] dashboard backed by a small set of static pages for the parts that need the Firebase web SDK directly (sign-in, account and device management, the developer/training console, and the privacy policy). It is hosted on Streamlit Community Cloud and deploys automatically from the repository's main branch.

10.1 A hybrid front end

Device registration needs the browser's Web Serial API, which cannot run in Streamlit's Python or its sandboxed component iframes, and authentication is easiest with the Firebase web SDK in the browser. The data views and charts, by contrast, are easiest in Streamlit's Python. Meowtion therefore serves both halves from the same origin so they share a login: the static sign-in page writes the Firebase ID token into a same-origin cookie, and the Python dashboard reads that cookie to identify the owner.

Table 11: Dashboard surfaces.

Surface	Tech	Role
`app.py`	Streamlit/Python	Live cat state, activity history, health analytics
`account.html`	JavaScript	Sign-in, account management, device pairing over Web Serial
`dev.html`	JavaScript	Developer/training console: capture, label, train, mode
`privacy.html`	JavaScript	Privacy policy

10.2 User experience and navigation

The product is built around a single hub, the static front door. From it the owner signs in to the dashboard, opens the developer console (if their account is enabled for it), or enters the read-only demo; logging out returns there. Figure 11 shows the navigation. A cohesive visual style, a white canvas with lavender accents and collapsible cards, is shared across every surface.

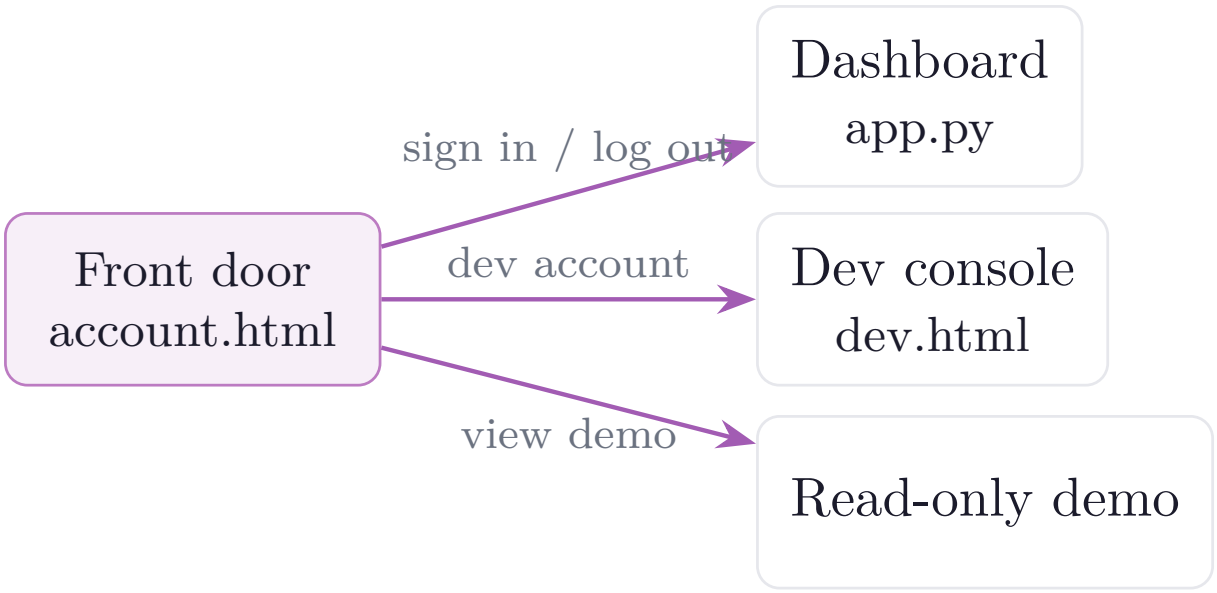


Figure 11: Dashboard page flow. The static front door is the hub for sign-in, the developer console, and the read-only demo.

10.3 Login and session

The owner logs in once on the front door with Firebase Auth; it writes the ID token into a same-origin cookie (`mtoken`). `app.py` reads that cookie client-side with the `extra-streamlit-components` cookie manager rather than the server-side cookie API, because Streamlit Community Cloud does not expose browser cookies to the server. Logged-out visitors get a sign-in gate and never see the dashboard; logging out navigates to the front door, signs out of Firebase, and clears the cookie. No token is ever placed in a URL. A separate flag cookie marks the public read-only demo.

10.4 Code structure

`app.py` is a thin entry point that only wires the pieces in order (theme, sign-in, live cards, analytics). Everything else is split into a `meowtion_dash` package so the analytics view can be edited without touching login or data-formatting code: `theme` (page config and brand header), `auth` (cookie/JWT to `(uid, token, is_demo)`), `firebase` (cached database reads), `data` (the clean activity DataFrame and filter helpers), `charts` (branded Altair charts), `live` (the live current-state cards), and `weather`. The editable analytics live in `dashboard_view.py`, which is handed a ready, one-row-per-event pandas DataFrame and never sees a cookie or raw Firebase JSON.

10.5 Live view versus history

The two reads have very different needs, and are split accordingly. The live current-state card refreshes every few seconds but needs only each cat's `current` record and its most recent events, so it issues a small per-cat read. The full activity history (potentially months of events) is fetched on a longer cache interval and reused across refreshes. Separating the two keeps the fast-refreshing live card from re-downloading the whole history on every tick, which keeps the page light and the backend bandwidth low.

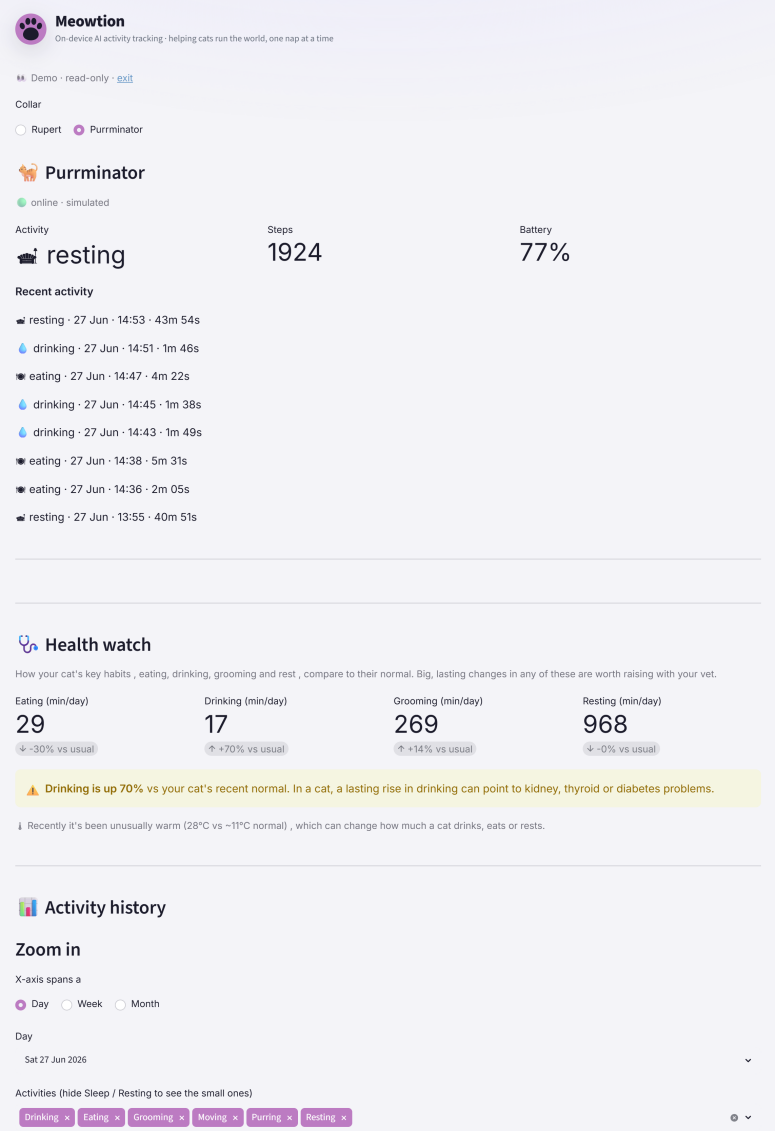


Figure 12: The dashboard (read-only demo): the collar switcher, the live current-state card, and the Health watch flagging a warm-weather rise in drinking.

10.6 Health analytics

The analytics view is built around health, not just a live read-out. A *health watch* compares each key habit (eating, drinking, grooming, resting) over recent days against the cat's own longer-run baseline, and flags a sustained change with a plain-language, vet-oriented note rather than a diagnosis. Two design choices keep it honest: it compares whole days only, since events are logged when an action ends and the current day is always partial; and it pairs a flagged change with local weather context, so a warm-spell rise in drinking is presented with its likely cause rather than as an alarm. A drill-down lets the viewer choose whether the x-axis spans a day, a week, or a month and then which specific window to show, and the activity charts are stacked by behaviour and rendered on a transparent background so they sit on the page's brand canvas.

10.7 Developer console

`dev.html` is visible only to developer accounts and is the data-collection and training control panel: live per-station capture status (signal, recording state, collar battery); the Training/Production mode switch; the editable list of actions to recognise; the labelled-clip review

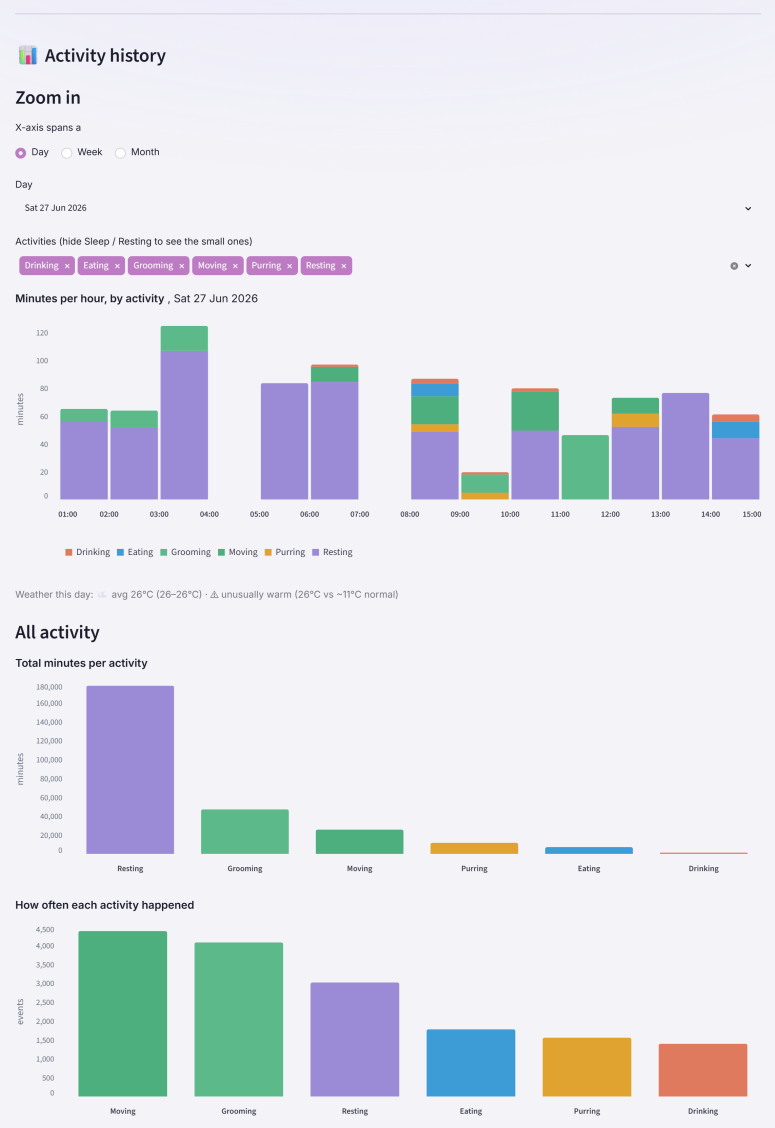


Figure 13: The history view (read-only demo): the day/week/month drill-down, the per-behaviour stacked chart, and the all-activity totals, with weather context.

with waveform trimming; dataset statistics and a motion-variety measure to gauge whether a class has enough varied examples; and the button that triggers cloud training. It also offers a full-screen, phone-style capture view with large tappable action buttons for live labelling at the bowl.

10.8 Read-only demo

A public demo lets anyone explore a populated product without an account. It reads the designated demo owner's data, which the security rules make world-readable, but every write control and function trigger is disabled and the dev console is available only in a read-only variant. The demo is genuinely read-only because it is enforced by the backend rules, not merely hidden in the interface (section 13).

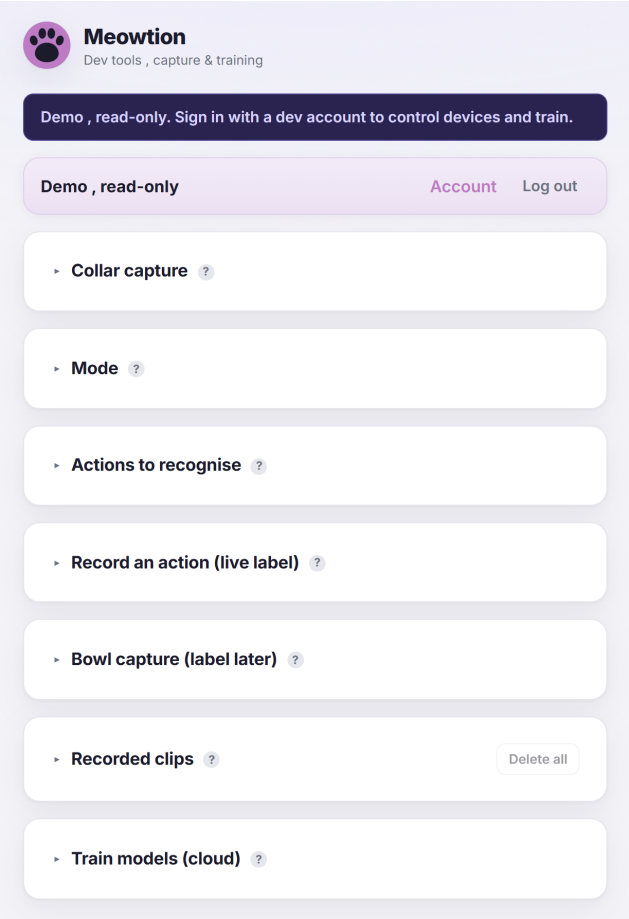


Figure 14: The developer console (read-only demo): the capture, mode, actions, clip-review, and cloud-training sections (cards collapsed by default).

11 User interface walkthrough

This section is a guided tour of the two interactive surfaces an operator uses, the sign-in front door and the developer console. It complements the implementation in section 10 by showing each screen and explaining what every control does. All screenshots are from the public read-only demo, so live device controls appear disabled and the private training audio is not shown.

11.1 Signing in and accounts

Figure 15 is the front door. An owner can sign in with email and password, or with **Continue with Google**. **Create one** opens registration, which records the owner's consent against the current privacy policy and sends a verification email; **Forgot password?** sends a reset link; and **View the demo (read-only)** enters the public demo with no account at all. Once signed in, the account page is where the owner pairs a station over Web Serial, registers or removes collars, and exercises the data-protection controls, exporting everything stored about their cats, or deleting the account and all its data (section 13). A linked privacy policy page states what is collected and why.

11.2 Moving through the app

Figure 16 shows the journey through the screens. The front door leads to the dashboard (by signing in, or read-only via the demo); from the dashboard a developer account can open the console, and any owner can reach the account page or log out. The console hosts the training

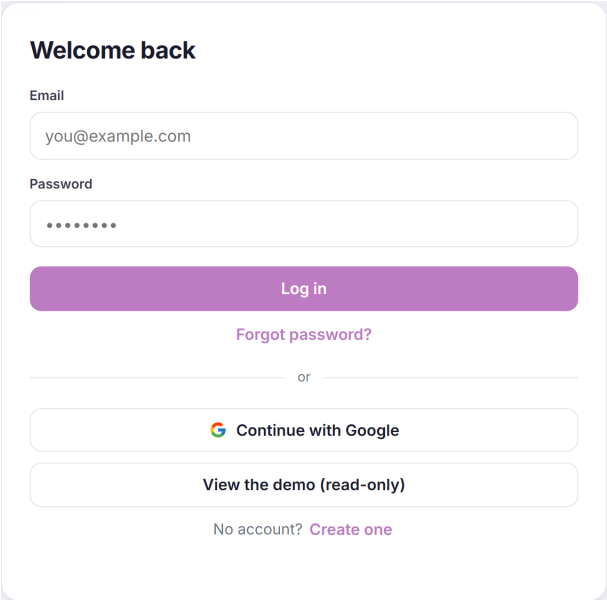


Figure 15: The sign-in front door: email/password, Google sign-in, the read-only demo, account creation, and password reset.

loop, capture, label, train, then switch to production, after which detections appear back on the dashboard.

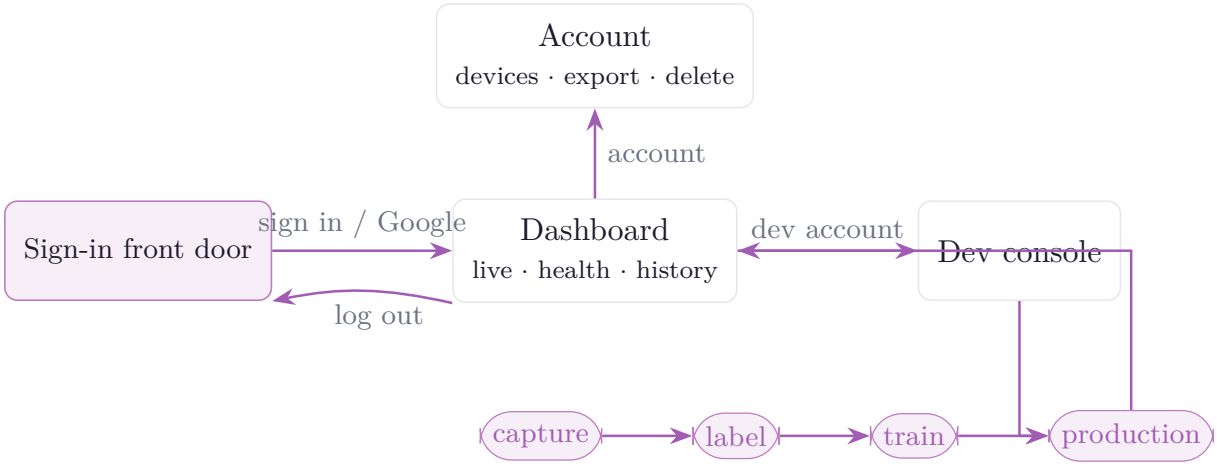


Figure 16: The user journey: front door to dashboard (or demo), the dashboard's branches to the account page and developer console, and the console's capture → label → train → production loop that feeds detections back to the dashboard.

11.3 The developer console

The developer console is where training data is collected and models are trained (sections 8 and 10). It is organised as collapsible cards, each with a `?` help popover; they are described below in order.

Collar capture (fig. 17). Live per-station status: which station and the collar in range, the signal strength (RSSI) and whether the collar is within the capture threshold, the station's power source, the collar's battery, how many collars are registered, and how long since the station was

last heard. This is how the operator confirms a collar is close enough to record before starting a capture.

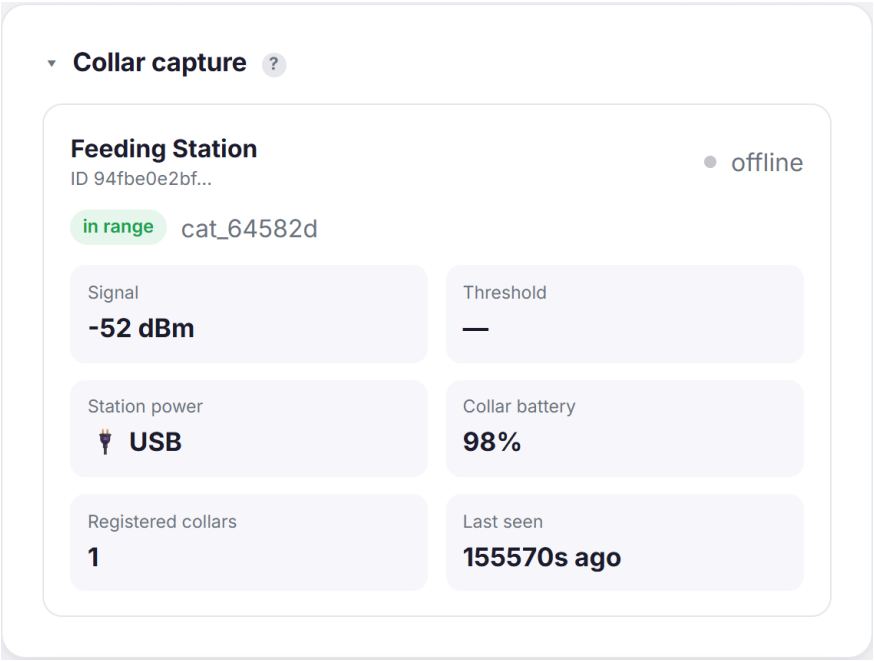


Figure 17: Collar capture: live station and collar status.

Mode (fig. 18). A single switch between **Training** (the station records and uploads clips) and **Production** (the station stops recording and only relays the collar's on-device classification). This is the mode referenced throughout section 3.

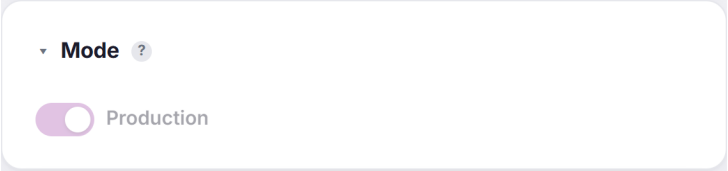


Figure 18: Mode: the Training / Production switch.

Actions to recognise (fig. 19). The editable list of behaviours the system should learn (here eat, drink, resting, moving). Adding or removing an action here changes both the labels offered when reviewing clips and the classes the cloud trainer learns, which is what makes the pipeline label-agnostic (section 8).

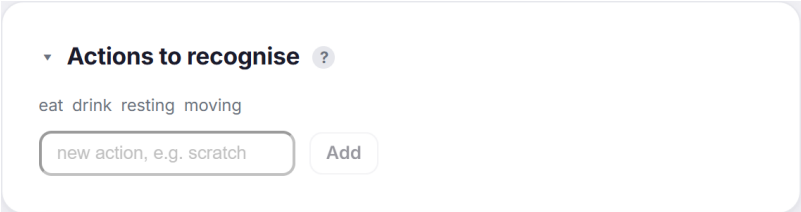


Figure 19: Actions to recognise: the owner-defined label set.

Record an action / live label (fig. 20). The fastest way to gather data: pick a clip length, press the button for what the cat is doing *now*, and the station records a clip at any distance and auto-labels it, so there is no labelling afterwards. A **Full screen** button gives a large, phone-style set of buttons for use at the bowl, and a clear warning appears when the station is offline (as shown) since clips cannot be captured without it.

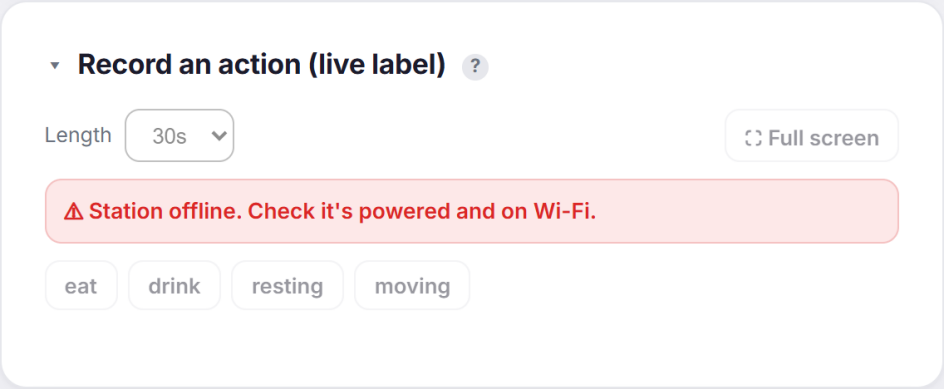


Figure 20: Record an action (live label): length, category buttons, full-screen mode, and the station-offline warning.

Bowl capture / label later (fig. 21). The alternative capture mode: with it on, the station records back-to-back whenever a registered collar is within a configurable signal-strength gate (with a dwell before it connects), and the clips are labelled afterwards in the clip review. This suits behaviours that happen reliably at the bowl.

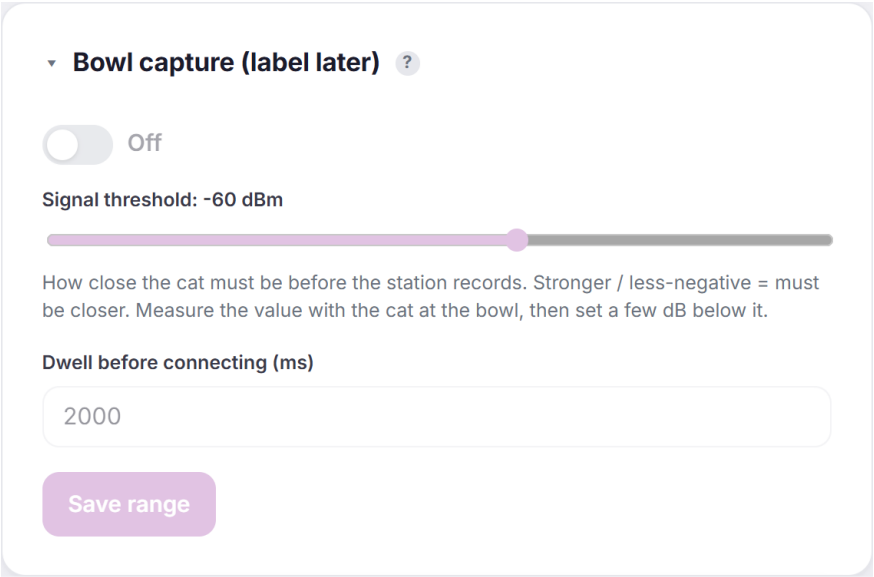


Figure 21: Bowl capture: the range gate (signal threshold and dwell) for unattended recording.

Recorded clips (fig. 22). The dataset, and the richest card. Clips are grouped by **day** and by **session** (a run of captures within the same few minutes), so a recording run stays together and is easy to organise. A per-label tally shows how many clips and sessions each action has, with a bar toward a recommended minimum, so the operator can see at a glance which classes need more data; **Analyse motion variety** scores how varied a label's samples are, a proxy for training quality (too-similar samples generalise poorly). Each clip row carries a play control to

hear the audio, a label dropdown, a **Motion** badge when time-aligned IMU data is present, its time and length, and a delete control. Clicking a clip expands it to a scrubber with the *audio waveform and the motion (accelerometer and gyroscope) traces drawn time-aligned beneath it*, plus draggable trim handles, so a sample can be auditioned against its movement and tidied before training. That expanded audio/motion view is hidden in the read-only demo because training audio is private to its owner.

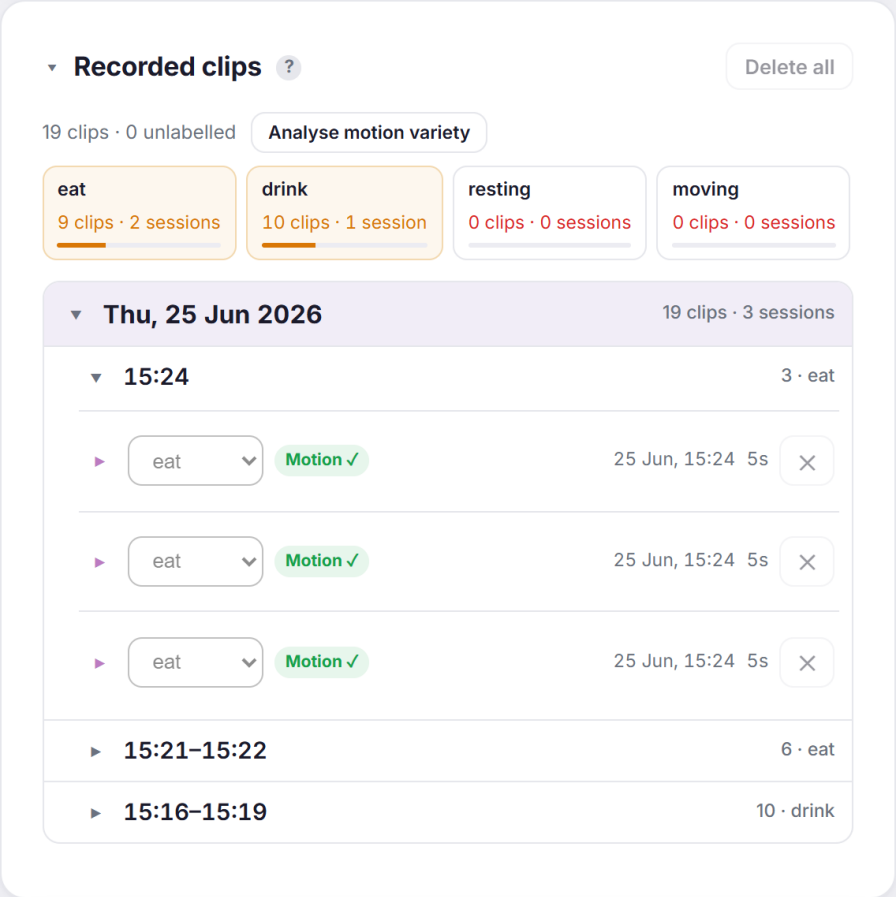


Figure 22: Recorded clips: per-label sample counts toward a minimum, the motion-variety quality check, and clips grouped by day and session, each with playback, a label, and a motion indicator.

Train models (cloud) (fig. 23). **Retrain models** triggers the `train` Cloud Function on the labelled clips (section 8); the line below reports live status, the current model version, and each stage's test accuracy and size (here an IMU stage and an audio stage). When training finishes, the new version is delivered to the collar over the air automatically (fig. 9).

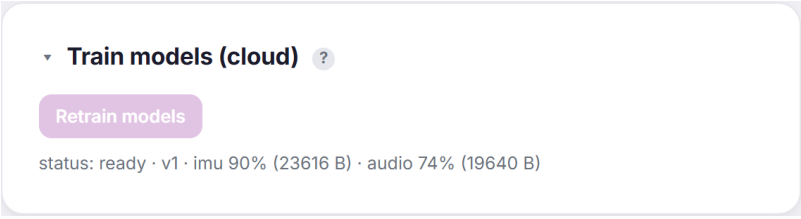


Figure 23: Train models (cloud): the retrain trigger and the live training status (version and per-stage accuracy).

12 Communication protocols

Four contracts tie the tiers together: the BLE telemetry the collar advertises, the audio clip-frame format it streams during training, the over-the-air model protocol, and the cloud HTTP it is all relayed over. The two BLE wire formats are defined once in shared headers (`firmware/common/meow_protocol.h` and `meow_ota.h`) that *both* firmwares include, so the collar (Zephyr) and station (ESP-IDF) builds cannot drift out of sync.

12.1 Telemetry advertisement

In production mode the collar publishes its result in the manufacturer-specific field of its BLE advertisement, so the station can read it passively without connecting. The packet is eight bytes (table 12); the collar's BLE address identifies which collar it is, so no identifier is carried in the payload. The `version` byte selects how the two result bytes are read: version 1 is the simulated behaviour state machine, version 2 is real on-device classification.

Table 12: Eight-byte telemetry packet (manufacturer AD data).

Bytes	Field	Meaning
0–1	company id	`0xFFFF` (test/development identifier)
2	version	`1` = simulated state, `2` = real on-device classification
3	state / class	v1: state (0 sleep …5 groom); v2: predicted class index (`0xFF` = unknown)
4	activity / confidence	v1: activity level 0–100; v2: confidence 0–100
5–6	steps	step count, `uint16`, little-endian
7	battery	battery level 0–100

Because the class is an index, not a name, the station and dashboard resolve it through the label list for the running model version, which keeps history correct across label-set changes (section 9).

12.2 Audio clip-streaming frame

While a station is subscribed in training mode, the collar streams continuously on the audio characteristic in 100 ms frames. Each frame is a packed header followed by an audio payload and a time-aligned IMU payload; the station concatenates frames and slices them into fixed-length training clips.

```
#define MEOW_CLIP_MAGIC 0x574F454Du   /* 'MEOW' little-endian, marks each frame
    start */

struct __attribute__((packed)) meow_clip_hdr {
    uint32_t magic;
    uint8_t  version;      /* 1 = 16-bit PCM audio, 2 = 8-bit G.711 mu-law */
    uint8_t  imu_axes;     /* 6 = accel x/y/z + gyro x/y/z */
    uint16_t imu_rate_hz;
    uint32_t audio_bytes;  /* audio payload size (one 100 ms frame) */
    uint32_t imu_bytes;    /* IMU payload size */
    uint32_t audio_rate;   /* output sample rate, for the station's WAV header */
};
/* frame layout:  [ meow_clip_hdr ][ audio payload ][ IMU payload ] */
```

Listing 4: Clip-frame header (`firmware/common/meow_protocol.h`).

To halve the data on the BLE link the collar encodes audio as 8-bit G.711 μ-law (`version` 2); the station decodes it back to 16-bit PCM for a normal WAV. The μ-law codec is defined inline in the same shared header, so the encode on the collar and the decode on the station are guaranteed to match, and this is the same representation the model is trained and run on (section 7).

12.3 GATT services

The collar exposes two custom 128-bit GATT services that share a common Meowtion base UUID: the *audio* service used for clip streaming, and the *OTA* service used for model delivery. The OTA service uses three UUIDs distinguished by a `0x0b**` group, one each for its control, data, and status characteristics. Both firmwares derive these UUIDs from the same header, the collar through Zephyr's encoding macro and the station from the raw little-endian byte arrays the header also provides.

12.4 Over-the-air model protocol

Model delivery is a small framed protocol over the OTA service. The client (station) writes opcodes to the *control* characteristic, streams the model to the *data* characteristic, and the collar acknowledges on a *status* notification. A transfer is: `BEGIN` (carrying the target slot, the total length, and a CRC), a stream of write-with-response `DATA` chunks (each chunk's write response is the flash-write flow control), then `END`. The collar erases the slot on `BEGIN`, writes chunks as they arrive, and on `END` re-reads the model from flash, verifies the CRC against the announced value, and reports the outcome.

```
/* CONTROL characteristic: first byte is an opcode; BEGIN is followed by
    meow_ota_begin. */
enum { MEOW_OTA_OP_BEGIN = 1, MEOW_OTA_OP_END = 2, MEOW_OTA_OP_ABORT = 3 };

struct __attribute__((packed)) meow_ota_begin {
    uint8_t  slot;       /* 0 = IMU model, 1 = audio model */
    uint32_t total_len;  /* model size in bytes */
    uint32_t crc32;      /* CRC-32/IEEE over the model blob */
};

/* STATUS characteristic (notify): the collar's acknowledgement. */
enum { MEOW_OTA_ST_READY = 1, MEOW_OTA_ST_RECEIVING = 2, MEOW_OTA_ST_OK = 3,
       MEOW_OTA_ST_ERR_SIZE = 5, MEOW_OTA_ST_ERR_CRC = 7, MEOW_OTA_ST_ERR_SEQ = 8
    };
```

Listing 5: OTA control opcodes, BEGIN payload, and status codes (`firmware/common/meow_ota.h`).

Crucially, `END` verifies and stages the model but does *not* swap the live model from inside the BLE callback. Committing a model is stack-heavy, and doing it in the radio callback crash-looped the device; the actual swap is deferred to a service call on the main thread's larger stack (section 5). That split, together with the raised main stack, is what made OTA reliable on hardware. A persisted slot magic marks a slot that holds a valid committed model, so the collar can tell an empty slot from a written one across a reboot. Figure 24 shows the message exchange.

12.5 Cloud transport

Above BLE, the station speaks ordinary HTTPS to Firebase: it writes telemetry to the Realtime Database through its REST interface, authorised by its device token, and POSTs training-clip bytes to the `upload_clip` Cloud Function with that same token (section 9). The dashboard is

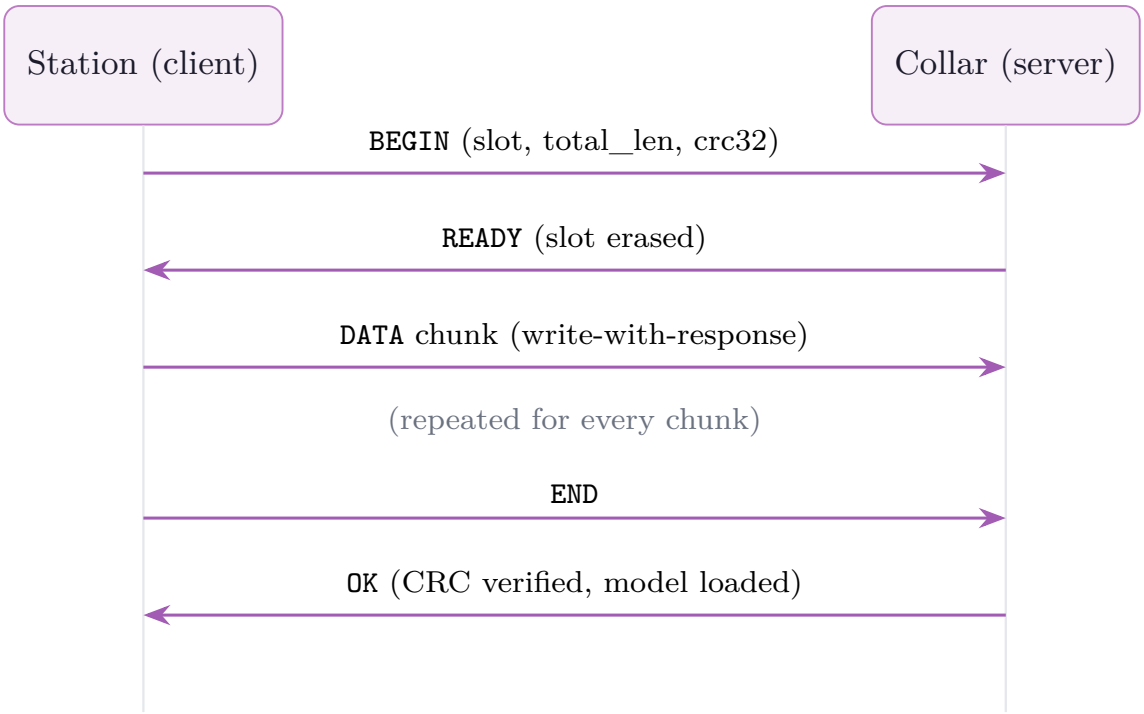


Figure 24: OTA model-push message sequence on the OTA service. The station writes `BEGIN`, streams the model in write-with-response `DATA` chunks, then writes `END`; the collar erases the slot, receives the bytes, verifies the CRC, loads the model, and acknowledges on the status characteristic.

a separate client of the same database. No bespoke transport or message broker is involved; the contract at this tier is just the database schema and the function's request format.

13 Security and privacy

Privacy is a design property of Meowtion, not a setting bolted on afterwards. The strongest guarantees come from where work happens (on the collar) and how data is scoped (per owner, by enforced rules).

13.1 Audio never leaves the collar

The microphone is used only for on-device classification. Audio is processed on the collar and discarded; it is never written to flash in normal operation and never transmitted in production mode. The collar's uplink is a classification result (a class index and a confidence), not a sensor stream. The one exception is deliberate and owner-driven: in training mode the owner chooses to capture clips to teach a behaviour, and those clips are stored privately to their own account and are never world-readable, even for the public demo.

13.2 Per-owner isolation

All of an owner's data lives under `users/<uid>/`, and the database and storage rules scope every read and write to the owning account, so owners (and their token-authenticated devices) touch only their own data. Isolation is enforced by the backend, not the client. The Storage rules go further and deny *all* direct client writes: training clips are written only by the `upload_clip` function and are readable only by their owning account, and model blobs are public-read (so the token-only station can fetch one) but writable only by the trainer.

13.3 Devices hold tokens, not credentials

A station authenticates with a scoped, revocable per-device token rather than the owner's password, and a collar holds nothing at all. A lost or compromised device therefore cannot expose the owner's credentials, and it can be revoked simply by deleting its token from the account, with no password change and no effect on the owner's other devices. Figure 25 traces the token's lifecycle.

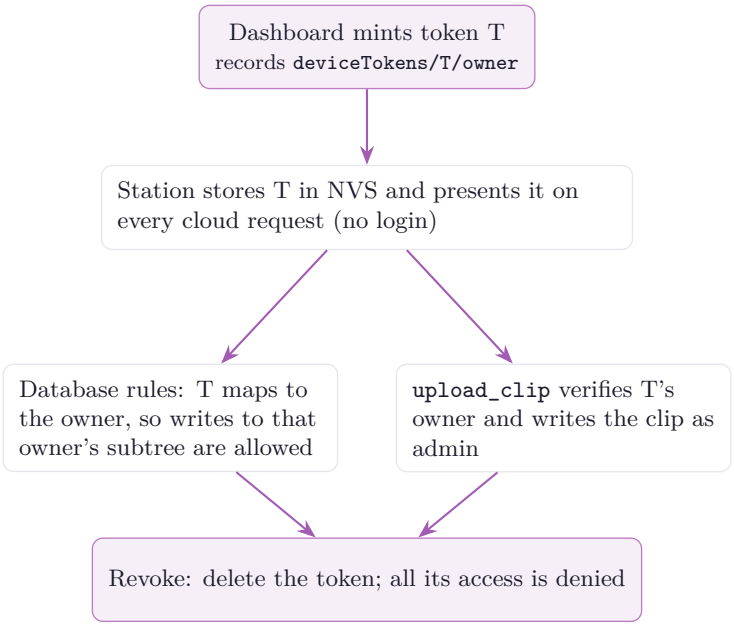


Figure 25: Credential-token lifecycle. A device is minted a scoped token that maps to its owner; the token authorises database writes and clip uploads, and deleting it revokes the device.

13.4 The demo is read-only by enforcement

The public demo reads a single designated demo account's data, which the rules make world-readable, but that account is write-locked at the backend: the demo cannot change any data or trigger any function. The read-only behaviour is enforced by the rules, not merely hidden in the interface, so it holds even against a hand-crafted client.

13.5 What is and is not a secret

The Firebase web API key shipped in the static front end is a public client identifier, not a secret; security is enforced by Authentication and the rules, not by hiding the key. No service-account key is present in the repository, and the functions run on their ambient service account. Device Wi-Fi credentials and firmware secrets are excluded by `.gitignore`. Everything required to understand and build the system is tracked; only true secrets are not.

13.6 Owner data rights

The account page lets an owner export everything stored about their cats and delete their account, devices, and data. This supports the access and erasure expectations of data-protection regimes such as the UK GDPR. The bundled privacy policy is a starting point and is marked for legal review before any public launch.

14 Evaluation and validation

This section reports the extent to which the system has been validated. It separates what has been *verified on real hardware*, what has been *build- or smoke-verified*, and what has *not yet been measured*, and it sets out the methodology for the measurements that remain. The intent is to be precise about the system's maturity rather than to imply results that have not been collected.

14.1 Validation status

The full pipeline, record a behaviour, label it, train models in the cloud, deliver them over the air, classify on the collar, and display the result, has been exercised end to end on real hardware: the collar reports an on-device class on its serial console and the relayed result reaches the dashboard. Table 13 summarises the status of each capability.

Table 13: Validation status by capability.

Capability	**Status**
On-device cascade inference (IMU stage)	Verified on hardware
Record → label → train → OTA → detect loop	Verified on hardware, end to end
Over-the-air model delivery	Verified on hardware
Collar firmware build (NCS v3.3.1)	Build-verified
Station firmware build (ESP-IDF)	Build-verified
Dashboard application	Smoke-verified (Streamlit app-test runs clean)
Audio confirmation stage of the cascade	Implemented; not yet active in production (IMU-only)
Telemetry state / activity / steps	Simulated until a model is loaded; real classifier output thereafter

14.2 Resource footprint

The collar is the binding resource constraint, and the firmware is built and reported against it. The tensor arenas are statically allocated and are the dominant RAM cost of inference; the audio window buffer and the BLE stack make up most of the rest. Enabling inference initially pushed RAM into the mid-90 % range; eliminating the float-window scratch buffer (quantising directly from int16, section 7) brought it down into the low-to-mid 80 % band of the 256 KiB part, which is where the current build sits. Flash use is well under half the application partition.

Table 14: Collar resource budget (the constraining device).

Resource	**Budget**	**Notes**
RAM (total)	256 KiB	Low-to-mid 80 % utilisation with inference enabled
IMU tensor arena	16 KiB	Always-on motion stage (starting budget)
Audio tensor arena	48 KiB	Confirmation stage (starting budget)
Model storage	256 KiB flash	Two OTA slots, executed in place (section 5)

14.3 What is not yet measured

The system has been validated for *function*, not yet for *performance*. The following metrics are not reported here because they have not been collected on a representative dataset or over a representative runtime; they are listed with the method by which each should be obtained, so the gap is explicit and the measurements are reproducible.

Table 15: Pending measurements and the method for each.

Metric	Method
Classification accuracy	Hold-out split of the labelled clips; report overall accuracy and a per-class confusion matrix for each cascade stage.
Cascade benefit	Compare IMU-only accuracy with the gated IMU+audio cascade on the ambiguous eat/drink pair; report how often, and how correctly, the audio stage fires.
Inference latency	Time `clf_classify` on-device across many windows; report mean and worst case for each stage.
Battery life	Run the collar to cutoff under a representative behaviour mix; report hours, and the sensitivity to the audio-stage duty cycle.
Power draw	Bench-measure current at the cell in each state (idle, IMU inference, audio inference, streaming, OTA).
OTA transfer time	Time a full model push over BLE; report bytes/s and total time per slot.

14.4 Threats to validity

Three caveats bound the current results. First, the behaviour data used so far is limited, so reported function does not yet imply field accuracy; a two-class model in particular will confidently label an idle cat as one of its two classes until an explicit idle/other class is added (section 15). Second, the demo dashboard is populated partly by a simulator (section 9), which is realistic but synthetic and must not be read as measured performance. Third, validation to date is single-cat and within-subject; multi-cat households are out of current scope. These are tracked as future work and do not affect the parts marked verified in table 13.

15 Future work

The system runs end to end on real hardware: record, label, train, deliver over the air, classify on-device, and display live. The following items are known and planned, roughly in priority order.

Idle/other class
Retrain with an explicit "neither" class so an idle cat is not forced into one of the trained behaviours. This is a data and modelling task, not a firmware change (section 8).

Activate the audio confirm stage
The cascade is built for IMU-then-audio confirmation; finish wiring the audio second stage into production so ambiguous motion is resolved by sound on-device (section 7).

Real classifier in telemetry
Switch the advertised telemetry from the simulated behaviour state machine to live classifier output once a trained model is loaded on the collar (section 12).

Battery characterisation
Measure real collar runtime across a representative behaviour mix and tune the duty cycle of the audio path against it.

More behaviours
Extend the captured and labelled set beyond eat/drink/purr to scratching, litter-tray use, grooming, and play, exercising the label-agnostic pipeline at scale.

Multi-cat disambiguation
Distinguish multiple cats in one household, currently framed as a single-cat, within-subject problem.

Firmware over-the-air
Today only models are delivered over the air; extend the mechanism to full firmware updates.

Publication
Write up the confidence-gated cascade as a short preprint and a venue paper.

A Bill of materials

The complete parts list for one collar and one station, plus the tools and print files. There is no custom PCB; each device is a Seeed XIAO module in a 3D-printed shell. Source of truth: `hardware/README.md`.

Table 16: Collar parts (one per cat).

Part	Qty	Notes
Seeed XIAO nRF52840 Sense	1	SoC with BLE; on-module IMU + PDM microphone; runs inference
1S LiPo cell, 3.7 V 100 mAh, with protection (BMS)	1	Soldered to the BAT pads; protected cell required
20 mm hydrophobic PTFE filter membrane	1	Over the mic hole; passes sound, blocks water/dust
10 mm quick-release safety collar	1	Standard strap; the TPU skin mounts to it

Table 17: Station parts (one per spot).

Part	Qty	Notes
Seeed XIAO ESP32-S3	1	BLE central + Wi-Fi gateway
USB-C cable + 5 V supply	1	Mains-powered; no battery

B Data-model reference

A consolidated reference for the cloud data model introduced in section 9: the Realtime Database tree, the Cloud Storage layout, and the request format of the two Cloud Functions.

B.1 Realtime Database tree

Table 18: Tools, consumables, and 3D-print files.

Item	Notes
USB-C cable	Flash and set up both boards
Filament	PETG for the rigid shells, flexible TPU for the collar skin
Water-resistant glue	Bonds the shell halves and seals against splashes
FDM 3D printer	A standard 0.4 mm nozzle is sufficient
`front-shell.stl`	Shared by collar and station (PETG)
`collar-back-shell.stl`	Collar (PETG)
`collar-silicone-skin.stl`	Collar (flexible TPU)
`station-back-shell.stl`	Station (any rigid filament)

```
users/<uid>/
  devices/
    <stationToken>/            type: "station"
      cats/<catId>/
        current                { ver, cls, conf, steps, battery, ts }
        events/<startMs>        { start, durationSec, type | cls, ver }
      weather/                  { current, history }
      config/                   { modelVer, mode, captureForce, ... }
      seen/<collarId>           unregistered collars heard nearby
    <collarId>/                 type: "collar"  (registry entry; name, simulated)
  models/                       { version, status, labels[], results }
  actions/                      behaviours to recognise (dev console)
  profile/                      { consent, location }
deviceTokens/<token>/owner      maps a device token to its owning account
config/
  demoOwner                     uid backing the public read-only demo
  devAccounts/<uid>             accounts allowed to use the dev/training tools
```

Listing 6: Realtime Database layout (owner-scoped).

Events store the model `version` and class `cls` index rather than a resolved name; the dashboard maps the index to a label through that version's `labels` list, so history stays correct across label-set changes.

B.2 Cloud Storage layout

```
training/<owner>/<collar>/<ts>.wav    training-clip audio   (private to owner; no
    client writes)
training/<owner>/<collar>/<ts>.imu    time-aligned motion   (private to owner)
models/<uid>/imu_model.tflite         trained IMU model     (public-read; written
    by train only)
models/<uid>/audio_model.tflite       trained audio model   (public-read; written
    by train only)
```

Listing 7: Cloud Storage layout.

B.3 Cloud Function requests

`train`

`POST` with header `Authorization: Bearer <Firebase ID token>` of a developer account. Trains from the account's labelled clips; returns the new version and label set. Gated additionally by `config/devAccounts/<uid>`.

`upload_clip`
: `POST` the raw clip bytes with the station's device token. The function resolves `deviceTokens/<token>/owner` and writes the file as administrator into that owner's Storage area. No Firebase login is involved.

C Glossary

`ATT/GATT`	Attribute Protocol / Generic Attribute Profile: the BLE layers that define characteristics and services.
`BLE`	Bluetooth Low Energy: the collar's only radio.
`BMS`	Battery Management System: the protection circuit on the LiPo cell.
`CMSIS-NN`	ARM's optimised neural-network kernels for Cortex-M cores.
`CRC`	Cyclic Redundancy Check; CRC-32/IEEE is used to verify an OTA model transfer.
`ESP-IDF`	Espressif IoT Development Framework: the station's SDK.
`IMU`	Inertial Measurement Unit: the collar's six-axis accelerometer + gyroscope.
`JWT`	JSON Web Token: the Firebase ID token format used for owner authentication.
`MTU`	Maximum Transmission Unit: the negotiated BLE payload size (247 bytes here).
`NCS`	nRF Connect SDK: the Nordic distribution of Zephyr used for the collar.
`NVS`	Non-Volatile Storage: the station's flash key/value store for provisioned credentials.
`OTA`	Over The Air: delivery of a trained model to the collar over BLE.
`PDM`	Pulse-Density Modulation: the collar microphone's output format.
`PHY`	Physical layer; the collar uses BLE's 2 Mbit PHY for throughput.
`RSSI`	Received Signal Strength Indication: used as the station's capture range gate.
`RTDB`	(Firebase) Realtime Database: the live state and history store.
`SoC`	System on Chip: the nRF52840 and ESP32-S3 modules.
`TFLM`	TensorFlow Lite Micro: the on-device inference runtime.
`UF2`	USB Flashing Format: the drag-and-drop image format for flashing the collar.
`XIP`	Execute In Place: running a model directly from flash without copying it to RAM.
μ`-law`	G.711 8-bit logarithmic audio companding, halving the BLE audio payload.

D References

[1] Zephyr Project. *Zephyr RTOS Documentation*. https://docs.zephyrproject.org. Accessed 2026.

[2] Espressif Systems. *ESP-IDF Programming Guide*. https://docs.espressif.com/projects/esp-idf. Accessed 2026.

[3] Robert David et al. 'TensorFlow Lite Micro: Embedded Machine Learning for TinyML Systems'. In: *Proceedings of Machine Learning and Systems (MLSys)* 3 (2021). arXiv: 2010.08678.

[4] Liangzhen Lai, Naveen Suda and Vikas Chandra. 'CMSIS-NN: Efficient Neural Network Kernels for Arm Cortex-M CPUs'. In: *arXiv preprint* (2018). arXiv: 1801.06601.

[5] Pete Warden and Daniel Situnayake. *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media, 2019. ISBN: 9781492052043.

[6] Google. *Firebase Documentation*. https://firebase.google.com/docs. Accessed 2026.

[7] Snowflake. *Streamlit Documentation*. https://docs.streamlit.io. Accessed 2026.